CU7 Performance Assessment — LangganKu (starter)

Fill in your name and the last four digits of your IC, then work through the Tasks of the paper **in this notebook**. First create your own data file in a terminal: `python make_my_data.py <L4>` (the file `subscriber_churn.csv` appears in this folder). Keep the notebook runnable from top to bottom.

In [1]:
```
import warnings; warnings.filterwarnings("ignore")
import mlflow, numpy as np, pandas as pd

NAME = "CHUA HANG JIE"      # e.g. "AISYAH BINTI AHMAD"
L4   = "0511"                # last four digits of your IC, keep the leading zero

# ---- your personal values (rules in the paper) ----
SEED      = int(L4)
digits = [int(d) for d in L4]
TEST_SIZE = 0.20 if digits[3] % 2 == 0 else 0.25
DEPTH = 3 + (digits[1] % 4)
C1        = 0.08
C2        = C1 * 10
RANK      = f"test_recall"
PORT      = 5111
EXP       = f"LangganKu_Churn_0511"
REG       = f"langganku_churn_0511"
```

Task 1 — Set-up, split and pipeline

In [2]:
```
# your code
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

mlflow.set_tracking_uri(f'sqlite:///langganku_0511.db')
mlflow.set_experiment(EXP)
print('Fracking URI:', mlflow.get_tracking_uri())

df = pd.read_csv('subscriber_churn.csv')
print(df.shape)
display(df.head())
```

```
2026/10/20 11:48:00 INFO mlflow.store.db.utils: Creating initial MLflow database tab
les...
2026/10/20 11:48:00 INFO mlflow.store.db.utils: Updating database tables
2026/10/20 11:48:03 INFO mlflow.tracking.fluent: Experiment with name 'LangganKu_Chu
rn_0511' does not exist. Creating a new experiment.
Fracking URI: sqlite:///langganku_0511.db
(1200, 10)
```

	subscriber_id	support_calls	payment_method	tenure_months	monthly_fee_rm	avg_hours
0	S0002	3	Bank transfer	28	55.0	
1	S0006	2	E-wallet	16	25.2	
2	S0008	2	Cash voucher	23	29.3	
3	S0010	1	E-wallet	34	50.0	
4	S0012	2	Bank transfer	20	40.9	

In [3]:
```
X= df.drop(columns=['subscriber_id', 'churned'])
y = df['churned']
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=TEST_SIZE,
    random_state=SEED
)

cat_cols = X.select_dtypes(exclude='number').columns.tolist()
num_cols = X.select_dtypes(include='number').columns.tolist()
print("categorical:", cat_cols); print("numeric:", num_cols)
print("train / test rows:", len(X_train), len(X_test))

def make_pipeline(model):
    pre = ColumnTransformer([('cat', OneHotEncoder(handle_unknown='ignore'), cat_co
                             remainder='passthrough')
    return Pipeline([('prep', pre), ("model", model)])
```

```
categorical: ['payment_method', 'plan', 'main_device', 'contract']
numeric: ['support_calls', 'tenure_months', 'monthly_fee_rm', 'avg_hours_week']
train / test rows: 900 300
```

Task 2 — Track the candidates in ONE experiment

In [4]:
```
# your code
import mlflow.sklearn
from mlflow.models import infer_signature
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score,recall_score, f1_score

def get_metrics(y_true, y_pred):
    return {
        'accuracy': float(accuracy_score(y_true, y_pred)),
        'precision': float(precision_score(y_true, y_pred, zero_division=0)),
        "recall": float(recall_score(y_true, y_pred)),
        'f1': float(f1_score(y_true, y_pred))
    }

candidates = [
    ('logreg-base', LogisticRegression(max_iter=1000),{'max_iter': 1000}, 'linear')
```

```
    (f"logreg-c{C1:g}", LogisticRegression(C=C1, max_iter=1000),{"C1":C1, "max_iter
    (f"logreg-c{C2:g}", LogisticRegression(C=C2, max_iter=1000),{"C2":C2, "max_iter
    (f"rf-d{DEPTH}", RandomForestClassifier(n_estimators=100, max_depth=DEPTH, rand
    {'n_estimators':100, "max_depth": DEPTH}, 'tree_ensemble')
]

run_ids = {}
for prefix, model, hparams, mtype in candidates:
    run_name = f"{prefix}-{L4}"
    with mlflow.start_run(run_name=run_name) as run:
        pipe = make_pipeline(model).fit(X_train, y_train)

        train_m = get_metrics(y_train, pipe.predict(X_train))
        test_m = get_metrics(y_test, pipe.predict(X_test))

        mlflow.log_params({
            'test_size': TEST_SIZE,
            'split_seed': SEED,
            'model': type(model).__name__, **hparams
        })
        mlflow.set_tags({
            'model_type': mtype,
            'candidate': NAME
        })
        mlflow.log_metrics({f"train_{k}": v for k, v in train_m.items()})
        mlflow.log_metrics({f'test_{k}': v for k, v in test_m.items()})

        sig = infer_signature(X_train, pipe.predict(X_train))
        mlflow.sklearn.log_model(
            pipe,
            name='model',
            signature=sig,
            input_example=X_train.head(3),
            skops_trusted_types=['sklearn.tree._tree.Tree']
        )

        run_ids[run_name] = run.info.run_id
        print(f"{run_name:<22} train={train_m}\n{'':<22} test={test_m}")
```

```
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
logreg-base-0511       train={'accuracy': 0.8477777777777777, 'precision': 0.7251908
396946565, 'recall': 0.4846938775510204, 'f1': 0.581039755351682}
                       test={'accuracy': 0.8433333333333334, 'precision': 0.76086956
52173914, 'recall': 0.49295774647887325, 'f1': 0.5982905982905983}
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
logreg-c0.08-0511      train={'accuracy': 0.8411111111111111, 'precision': 0.7523809
523809524, 'recall': 0.4030612244897959, 'f1': 0.5249169435215947}
                       test={'accuracy': 0.84, 'precision': 0.7804878048780488, 'rec
all': 0.4507042253521127, 'f1': 0.5714285714285714}
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
logreg-c0.8-0511       train={'accuracy': 0.8455555555555555, 'precision': 0.7209302
325581395, 'recall': 0.4744897959183674, 'f1': 0.5723076923076923}
                       test={'accuracy': 0.8433333333333334, 'precision': 0.76086956
52173914, 'recall': 0.49295774647887325, 'f1': 0.5982905982905983}
Downloading artifacts:   0%|          | 0/7 [00:00<?, ?it/s]
```

```
rf-d4-0511            train={'accuracy': 0.8188888888888889, 'precision': 0.9714285
714285714, 'recall': 0.17346938775510204, 'f1': 0.2943722943722944}
                      test={'accuracy': 0.8033333333333333, 'precision': 1.0, 'reca
ll': 0.16901408450704225, 'f1': 0.2891566265060241}
```

Task 3 — Choose the best run from code and justify it

In [5]:
```
#### your code
runs = mlflow.search_runs(
    experiment_names=[EXP],
    filter_string='attributes.status = "FINISHED"',
    order_by=["metrics.test_recall DESC"]
)
runs['gap_recall'] = (runs['metrics.train_recall'] - runs['metrics.test_recall']).a
display(runs[[
    'tags.mlflow.runName',
    'tags.model_type',
    'metrics.test_f1',
    'metrics.test_recall',
    'metrics.test_precision',
    'metrics.test_accuracy',
    'metrics.train_f1',
    'gap_recall'
]])

best = runs.iloc[0]
BEST_RUN_ID = best['run_id']
BEST_NAME = best['tags.mlflow.runName']
BASE_NAME = candidates[0][0] + f"-{L4}"
BASE_RUN_ID = run_ids[BASE_NAME]
print(f"Best run: {BEST_NAME}  (test_recall = {best['metrics.test_recall']:.4f})")
```

	tags.mlflow.runName	tags.model_type	metrics.test_f1	metrics.test_recall	metrics.test_prec
0	logreg-c0.8-0511	linear	0.598291	0.492958	0.76
1	logreg-base-0511	linear	0.598291	0.492958	0.76
2	logreg-c0.08-0511	linear	0.571429	0.450704	0.78
3	rf-d4-0511	tree_ensemble	0.289157	0.169014	1.00

```
Best run: logreg-c0.8-0511  (test_recall = 0.4930)
```

Why I chose this run (2–3 sentences):

- I ranked the runs from code by **test_recall** (Higher is better).
- Model run name `logreg-c0.8-0511` came first when I arrange them in decending order (Higher to lowest), with the **test_recall** score around **0.4930** (4 decimal place). Based on the data, it's value is identical to run name `logreg-base-0511`. But the reason I choose the first one is because the `gap_recall` is smaller.

Task 4 — Register, alias and roll back

In [11]:
```
# your code
from mlflow import MlflowClient
client = MlflowClient()

v1 = mlflow.register_model(f"runs:/{BASE_RUN_ID}/model", REG)
client.set_registered_model_alias(REG, 'champion', v1.version)
print('v1 =', BASE_NAME, '-> champion')


v2 = mlflow.register_model(f"runs:/{BEST_RUN_ID}/model",REG)
client.set_registered_model_alias(REG, 'champion', v2.version)
client.set_model_version_tag(REG, v2.version, 'approved_by', NAME)
client.update_model_version(
    REG, v2.version,
    description=f"{BEST_NAME}: best test_recall = {best['metrics.test_recall']:.4f}
                f"(split seed {SEED}, test_size {TEST_SIZE}). Promoted over baselin
)
print('v2 = ', BEST_NAME, '-> champion')


client.set_registered_model_alias(REG, 'champion', v1.version)
print('ROLLACK champion ->', client.get_model_version_by_alias(REG, 'champion').ver


client.set_registered_model_alias(REG, 'champion', v2.version)
mv = client.get_model_version_by_alias(REG, 'champion')
print(f'FINAL    champion -> v{mv.version} | run {mv.run_id} | tags {mv.tags}')
```

```
Registered model 'langganku_churn_0511' already exists. Creating a new version of th
is model...
2026/10/20 11:50:22 WARNING mlflow.tracking._model_registry.fluent: Run with id 6edf
b16dc0f34ff0bd8d2fd4dc42026f has no artifacts at artifact path 'model', registering
model based on models:/m-7d63f4a173874cdcadab2010442018de instead
Created version '3' of model 'langganku_churn_0511'.
Registered model 'langganku_churn_0511' already exists. Creating a new version of th
is model...
2026/10/20 11:50:22 WARNING mlflow.tracking._model_registry.fluent: Run with id d1fe
5a9c496342b09372e911ac014ecf has no artifacts at artifact path 'model', registering
model based on models:/m-4851f909344a4e44914d507b98f14680 instead
Created version '4' of model 'langganku_churn_0511'.
v1 = logreg-base-0511 -> champion
v2 =  logreg-c0.8-0511 -> champion
ROLLACK champion -> 3
FINAL    champion -> v4 | run d1fe5a9c496342b09372e911ac014ecf | tags {'approved_b
y': 'CHUA HANG JIE'}
```

Task 5 — Serve the champion and call it

In [12]:
```
# your code
import os, sys, time, subprocess, requests
```

```
MODEL_URI = f"models:/{REG}@champion"
URL = f"http://127.0.0.1:{PORT}"

def server_up():
    try:    return requests.get(URL + "/ping", timeout=2).status_code == 200
    except requests.exceptions.RequestException: return False

server = None
if not server_up():
    env = {**os.environ, "MLFLOW_TRACKING_URI": f"sqlite:///langganku_{L4}.db"}
    server = subprocess.Popen([sys.executable, "-m", "mlflow", "models", "serve", "
                               "-p", str(PORT), "--env-manager", "local"], env=env)
    for _ in range(90):                      # wait up to ~90 s for the server to st
        if server_up(): break
        time.sleep(1)
print("server up:", server_up(), "on", URL)
```

```
server up: True on http://127.0.0.1:5111
```

In [13]:

```
# "Your row": row number L4 mod n of the feature table (the same row the checker us
row = X.iloc[[int(L4) % len(X)]]
display(row)

payload = {"dataframe_split": row.to_dict(orient="split", index=False)}
resp = requests.post(URL + "/invocations", json=payload, timeout=30)
print("HTTP", resp.status_code, resp.json())

pred = resp.json()["predictions"][0]
local = mlflow.pyfunc.load_model(MODEL_URI).predict(row)[0]
print(f"server predicted class (1 = will default): {pred}   | same model loaded loc
```

	support_calls	payment_method	tenure_months	monthly_fee_rm	avg_hours_week	plan
511	1	E-wallet	9	28.9	7.2	Basic

```
HTTP 200 {'predictions': [0]}
server predicted class (1 = will default): 0   | same model loaded locally: 0
```

In [ ]: